WEB & APP DEVELOPMENT

React

Build declarative UIs with components, JSX, and hooks — from props/state and useEffect to lists, forms, data fetching, routing, and performance.

react · jsx · hooks · useState · useEffect · components · performance

Study Pack · Comprehensive reference & interview prep

CONTENTS

1. What React Is & Why It Matters
2. Setup & Getting Started
3. Components & JSX
4. Props & State
5. Hooks — The Essential Set
6. Lists, Keys & Conditional Rendering
7. Forms & Controlled Components
8. Lifting State Up & Data Flow
9. Data Fetching
10. Routing Overview
11. Performance Optimisation
12. Cheat-Sheet & Hook Reference
13. Best Practices & Common Pitfalls
14. Interview Questions & Answers
15. Further Learning

What React Is & Why It Matters

React is a declarative, component-based JavaScript library for building user interfaces. You describe *what* the UI should look like for a given state, and React efficiently updates the DOM to match whenever that state changes.

Created at Facebook and open-sourced in 2013, React is the most widely used UI library on the web. Its core idea is simple but powerful: your UI is a pure function of state, `UI = f(state)`. You never manually touch the DOM to insert or remove nodes — you change a value, and React figures out the minimal set of DOM mutations needed.

DECLARATIVE

Describe the target UI for each state; React handles the imperative DOM updates. Fewer bugs than hand-written `document.createElement` code.

COMPONENT-BASED

Build encapsulated, reusable pieces that manage their own state, then compose them into complex UIs like LEGO bricks.

LEARN ONCE, WRITE ANYWHERE

The same mental model powers the web (`react-dom`), native mobile (React Native), and server rendering.

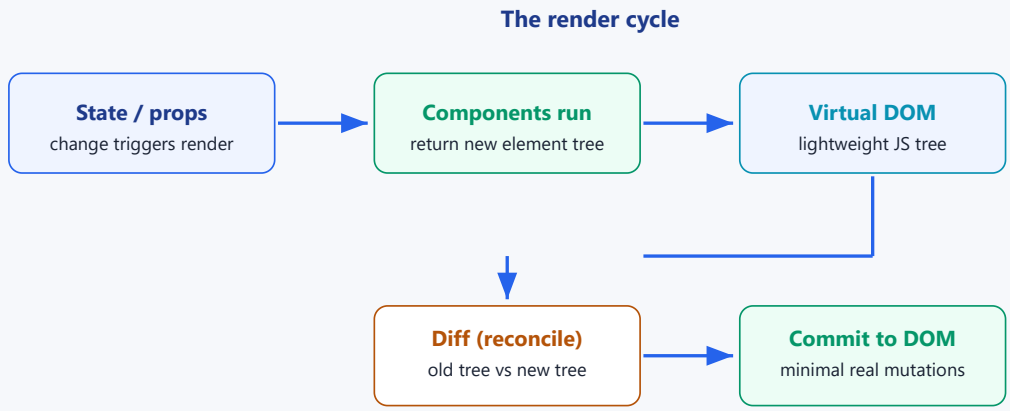


Fig 1 — A state change re-runs components to produce a new virtual DOM, which React diffs against the previous one and commits only the minimal real-DOM changes.

> **Virtual DOM**
> The virtual DOM is just an in-memory tree of plain JS objects describing your UI. Comparing two JS trees is far cheaper than reading/writing the real DOM, so React batches and minimises actual DOM work.

Setup & Getting Started

The old `create-react-app` tool is deprecated. In 2026 the recommended paths are **Vite** for a client-side single-page app, or a framework like **Next.js** for server rendering and routing out of the box.

BASH

```
# --- Vite (fast, minimal SPA) ---
npm create vite@latest my-app -- --template react
cd my-app
npm install
npm run dev          # local dev server with hot module replacement
npm run build        # production bundle into dist/

# --- Next.js (full framework: routing, SSR, server components) ---
npx create-next-app@latest my-next-app

# React itself is just two packages:
npm install react react-dom
```

The entry point mounts your root component into a DOM node using the React 18+ `createRoot` API:

JSX

```jsx
// main.jsx — the application entry point
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";
import "./index.css";

// StrictMode double-invokes renders in dev to surface impure logic.
createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

> ✦ **Tip**
> StrictMode intentionally runs effects and renders twice in development only. If something breaks under it, your code has a hidden side effect — fix the code rather than removing StrictMode.

Components & JSX

A component is a JavaScript function that returns markup. Component names **must start with a capital letter** — React treats lowercase tags as built-in HTML elements.

JSX

```jsx
// A function component returns JSX describing the UI.
function Greeting() {
  return <h1>Hello, world!</h1>;
}

// Components compose: use one inside another like a custom tag.
function App() {
  return (
    <main>
      <Greeting />
      <Greeting />
    </main>
  );
}
```

JSX is syntax sugar that compiles to `React.createElement` calls. It looks like HTML but it is JavaScript, so a few rules differ:

ONE ROOT ELEMENT

A component must return a single parent. Wrap siblings in a `<div>` or an empty **Fragment** `<> … </>` to avoid extra DOM nodes.

CAMELCASE ATTRIBUTES

`className` not `class`, `htmlFor` not `for`, `onClick` not `onclick`. Styles are objects: `style={{color:'red'}}`.

CURLY BRACES EMBED JS

Any JS expression goes inside `{ }` — variables, function calls, ternaries. Statements like `if` or `for` are not allowed inside JSX.

CLOSE EVERY TAG

JSX is XML-strict: self-closing tags need a slash, e.g. `<img />`, `<br />`.

JSX

```jsx
function Profile() {
  const user = { name: "Ada", avatar: "/ada.png" };
  const isOnline = true;

  return (
    <>
      {/* curly braces embed JS expressions */}
      <img src={user.avatar} alt={user.name} className="avatar" />
      <h2>{user.name}</h2>
      <span>{isOnline ? "🟢 Online" : "⚪ Offline"}</span>
    </>
  );
}
```

Props & State

Data flows one way in React: **down**, from parent to child, via **props**. Props are read-only inputs — a component must never mutate its own props. **State** is private, mutable-over-time data owned by a component.

Props

JSX

```jsx
// Props arrive as a single object; destructure for readability.
function UserCard({ name, role, verified = false }) {
  return (
    <div className="card">
      <h3>{name} {verified && "✓"}</h3>
      <p>{role}</p>
    </div>
  );
}

// Parent passes props as attributes. `children` is a special prop
// holding whatever you nest between the tags.
function App() {
  return <UserCard name="Linus" role="Maintainer" verified />;
}
```

children

Content placed between a component's tags is passed as the `children` prop, enabling flexible wrapper components like `<Panel> ... </Panel>`.

State with useState

JSX

```jsx
import { useState } from "react";

function Counter() {
  // useState returns [currentValue, setterFunction].
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}
```

State is a snapshot

Calling `setCount` does not change `count` immediately — it schedules a re-render. The `count` variable keeps its value for the rest of the current render. To update based on the previous value, pass an **updater function**.

JSX

```jsx
// WRONG: all three see the same stale snapshot — count only goes up by 1.
setCount(count + 1);
setCount(count + 1);
setCount(count + 1);

// RIGHT: updater form receives the latest pending value — goes up by 3.
setCount(c => c + 1);
setCount(c => c + 1);
setCount(c => c + 1);
```

Never mutate state objects or arrays directly. Create a new reference so React detects the change:

JSX

```jsx
const [todos, setTodos] = useState([]);

// WRONG — mutates existing array, same reference, no re-render guaranteed
todos.push(newTodo);
setTodos(todos);

// RIGHT — new array via spread
setTodos([...todos, newTodo]);

// Updating one item immutably
setTodos(todos.map(t => t.id === id ? { ...t, done: true } : t));

// Removing an item
setTodos(todos.filter(t => t.id !== id));
```

Hooks — The Essential Set

Hooks are functions starting with `use` that let function components "hook into" React features. Two rules govern all hooks:

> ✕ **Rules of Hooks**
> (1) Only call hooks at the **top level** of a component or custom hook — never inside loops, conditions, or nested functions. (2) Only call them from React function components or other hooks. React relies on a stable call order to associate each hook with its state.

useEffect — synchronising with the outside world

Effects run *after* render to synchronise with external systems: network, subscriptions, timers, direct DOM APIs. The dependency array controls when the effect re-runs.

JSX

```jsx
import { useState, useEffect } from "react";

function Clock() {
  const [now, setNow] = useState(new Date());

  useEffect(() => {
    const id = setInterval(() => setNow(new Date()), 1000);
    // The returned cleanup runs before the next effect and on unmount.
    return () => clearInterval(id);
  }, []); // empty deps = run once after mount

  return <time>{now.toLocaleTimeString()}</time>;
}
```



Fig 2 — useEffect lifecycle: the effect runs after mount, re-runs (cleanup → effect) when a dependency changes, is skipped when deps are unchanged, and cleans up on unmount.

> ⚠ **Dependency array**
>
> Omitting the array runs the effect after *every* render. An empty `[]` runs it once. List every reactive value the effect reads — the lint rule `react-hooks/exhaustive-deps` enforces this. Lying about deps causes stale closures and bugs.

useRef — mutable boxes & DOM access

JSX

```jsx
import { useRef, useEffect } from "react";

function SearchBox() {
  const inputRef = useRef(null); // { current: null }

  useEffect(() => {
    inputRef.current.focus(); // imperative DOM access after mount
  }, []);

  return <input ref={inputRef} placeholder="Search..." />;
}
```

A ref holds a mutable value in `.current` that persists across renders **without triggering a re-render** when changed. Use it for DOM nodes, timer IDs, or any "instance variable".

useContext — avoiding prop drilling

JSX

```jsx
import { createContext, useContext } from "react";

const ThemeContext = createContext("light");

function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Toolbar />
    </ThemeContext.Provider>
  );
}

function Toolbar() {
  // Reads the nearest Provider's value — no props threaded through.
  const theme = useContext(ThemeContext);
  return <div className={theme}>Current theme: {theme}</div>;
}
```

useMemo & useCallback — caching between renders

JSX

```jsx
import { useMemo, useCallback } from "react";

function ProductList({ products, query }) {
  // Recompute the filtered list only when products or query change.
  const visible = useMemo(
    () => products.filter(p => p.name.includes(query)),
    [products, query]
  );

  // Stable function identity so memoised children don't re-render.
  const handleBuy = useCallback((id) => {
    console.log("buy", id);
  }, []);

  return visible.map(p =>
    <Item key={p.id} product={p} onBuy={handleBuy} />
  );
}
```

> **Mental model**
>
> `useMemo` caches a *computed value*; `useCallback` caches a *function*. `useCallback(fn, deps)` is just `useMemo(() => fn, deps)`. Reach for them only to fix a measured performance problem, not by default.

Custom hooks — extracting reusable logic

JSX

```
// Any function that calls hooks and starts with "use" is a custom hook.
function useLocalStorage(key, initial) {
  const [value, setValue] = useState(() => {
    const stored = localStorage.getItem(key);
    return stored ? JSON.parse(stored) : initial;
  });

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue];
}

// Usage reads just like useState but persists across reloads.
function Settings() {
  const [name, setName] = useLocalStorage("name", "");
  return <input value={name} onChange={e => setName(e.target.value)} />;
}
```

Lists, Keys & Conditional Rendering

Render collections by mapping an array to JSX. Each sibling needs a stable, unique `key` so React can track items across renders.

JSX

```
function TodoList({ todos }) {
  if (todos.length === 0) {
    return <p>Nothing to do 🎉</p>;   // early-return conditional
  }

  return (
    <ul>
      {todos.map(todo => (
        <li key={todo.id}>
          {todo.done ? <s>{todo.text}</s> : todo.text}
        </li>
      ))}
    </ul>
  );
}
```

✕ **Never use the array index as key**
when the list can reorder, insert, or delete — React will associate state with the wrong item, causing subtle bugs (wrong input values, lost focus). Use a stable domain ID.

Common conditional patterns:

Pattern	Code	Use when
Ternary	`{ok ? <A/> : <B/>}`	Choose between two outputs
Logical AND	`{ok && <A/>}`	Render or render nothing
Early return	`if (!data) return <Spinner/>;`	Guard clauses at top of component
Return null	`return null;`	Render nothing from a component

⚠ **The `0` gotcha**
`{count && <Badge/>}` renders the literal `0` when count is zero, because `0` is falsy but still a renderable value. Use `{count > 0 && <Badge/>}` instead.

Forms & Controlled Components

In a **controlled component**, React state is the single source of truth: the input's `value` comes from state, and `onChange` writes back to state.

JSX

```jsx
import { useState } from "react";

function SignupForm() {
  const [form, setForm] = useState({ email: "", agree: false });

  function handleChange(e) {
    const { name, value, type, checked } = e.target;
    setForm(prev => ({
      ...prev,
      [name]: type === "checkbox" ? checked : value,
    }));
  }

  function handleSubmit(e) {
    e.preventDefault();              // stop full-page reload
    console.log("submit", form);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        name="email"
        type="email"
        value={form.email}
        onChange={handleChange}
      />
      <label>
        <input
          name="agree"
          type="checkbox"
          checked={form.agree}
          onChange={handleChange}
        />
        I agree
      </label>
      <button disabled={!form.agree}>Sign up</button>
    </form>
  );
}
```

✦ **Controlled vs uncontrolled**

Controlled inputs give you instant validation and derived UI. For simple forms, **uncontrolled** inputs read via a `ref` or the native form API are lighter. Libraries like React Hook Form combine both for performance.

Lifting State Up & Data Flow

When two components need to share state, move it to their closest common ancestor and pass it down as props. The ancestor owns the state; children receive values and callbacks. This keeps a single source of truth.

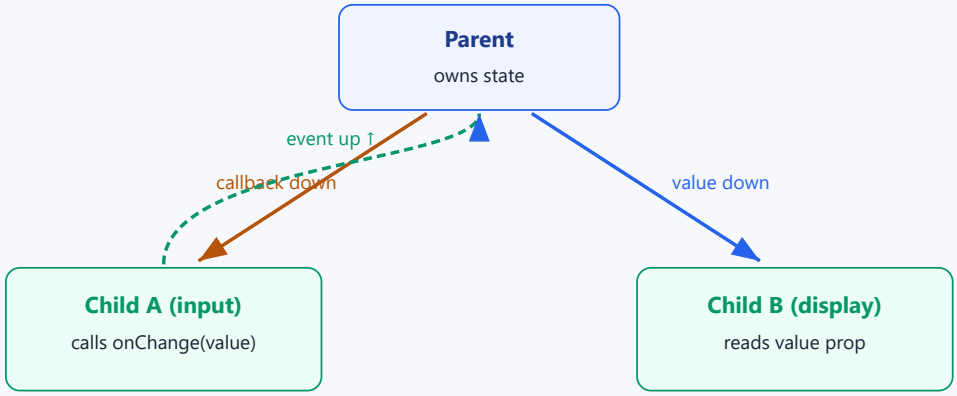


Fig 3 — Unidirectional data flow: state lives in the common parent, flows down as props, and child events flow up through callbacks to update it.

> **When context/stores aren't enough**
> For deeply shared or server state, reach for `useReducer` + context, or dedicated libraries: Redux Toolkit / Zustand for client state, TanStack Query for server cache.

Data Fetching

In a classic SPA you fetch in an effect, tracking loading and error states. Always guard against race conditions when the component unmounts or the request key changes.

JSX

```
import { useState, useEffect } from "react";

function useUser(id) {
  const [state, setState] = useState({ status: "loading", data: null, error: null });

  useEffect(() => {
    let ignore = false;             // guard against stale responses
    const controller = new AbortController();

    setState({ status: "loading", data: null, error: null });
    fetch(`/api/users/${id}`, { signal: controller.signal })
      .then(r => {
        if (!r.ok) throw new Error(`HTTP ${r.status}`);
        return r.json();
      })
      .then(data => { if (!ignore) setState({ status: "ok", data, error: null }); })
      .catch(err => {
        if (err.name !== "AbortError" && !ignore)
          setState({ status: "error", data: null, error: err });
      });

    return () => { ignore = true; controller.abort(); };
  }, [id]);

  return state;
}
```

✦ **Prefer a data library**

Hand-rolled fetching gets repetitive and error-prone. **TanStack Query** (React Query) handles caching, deduping, retries, background refetch, and stale-while-revalidate in a few lines. In frameworks like Next.js, fetch in Server Components instead.

JSX

```jsx
// The same thing with TanStack Query — caching & states for free.
import { useQuery } from "@tanstack/react-query";

function Profile({ id }) {
  const { data, isPending, error } = useQuery({
    queryKey: ["user", id],
    queryFn: () => fetch(`/api/users/${id}`).then(r => r.json()),
  });

  if (isPending) return <Spinner />;
  if (error) return <p>Failed: {error.message}</p>;
  return <h2>{data.name}</h2>;
}
```

Routing Overview

React has no built-in router. The two dominant choices in 2026 are **React Router** (for Vite SPAs) and the **Next.js App Router** (file-system based).

JSX

```jsx
// React Router v6/v7 — declarative routes
import { createBrowserRouter, RouterProvider, Link, useParams } from "react-router-dom

function User() {
  const { id } = useParams();        // read URL params
  return <h2>User {id}</h2>;
}

const router = createBrowserRouter([
  { path: "/", element: <Home /> },
  { path: "/users/:id", element: <User /> },
  { path: "*", element: <NotFound /> },
]);

function App() {
  return <RouterProvider router={router} />;
}

// Navigation uses <Link> (not <a>) to avoid full page reloads.
<Link to="/users/42">View user</Link>
```

Concern	React Router	Next.js App Router
Route definition	Config array / JSX	Folder & file names
Rendering	Client-side (SPA)	Server Components + SSR default
Data loading	Loaders / fetch in effect	`async` server components
Best for	Dashboards, internal apps	SEO sites, full-stack apps

Performance Optimisation

React is fast by default; most "performance problems" are unnecessary re-renders. Optimise only after profiling with the React DevTools Profiler.

REACT.MEMO

Wraps a component to skip re-render when its props are shallow-equal to the previous render. Pair with `useCallback` / `useMemo` for stable prop identities.

KEYS & STRUCTURE

Stable keys prevent remounting. Moving state down or lifting content up (`children`) avoids re-rendering whole subtrees.

CODE SPLITTING

`lazy()` + `<Suspense>` defer loading of routes/components until needed, shrinking the initial bundle.

LIST VIRTUALISATION

Render only visible rows of huge lists with TanStack Virtual or react-window.

JSX

```jsx
import { memo, lazy, Suspense } from "react";

// Only re-renders when `row` actually changes.
const Row = memo(function Row({ row }) {
  return <li>{row.label}</li>;
});

// Lazy-load a heavy route; show a fallback while the chunk downloads.
const Dashboard = lazy(() => import("./Dashboard.jsx"));

function App() {
  return (
    <Suspense fallback={<Spinner />}>
      <Dashboard />
    </Suspense>
  );
}
```

✓ **React Compiler**

As of 2026 the React Compiler can auto-memoise components and hooks at build time, often removing the need for manual `useMemo` / `useCallback` / `memo`. Adopt it and lean on it before hand-tuning.

Cheat-Sheet & Hook Reference

Hook	Signature	Purpose
`useState`	`[v, setV] = useState(init)`	Local reactive state
`useEffect`	`useEffect(fn, deps)`	Side effects / external sync
`useContext`	`v = useContext(Ctx)`	Read context value
`useRef`	`ref = useRef(init)`	Mutable box / DOM node
`useMemo`	`v = useMemo(fn, deps)`	Cache computed value
`useCallback`	`fn = useCallback(fn, deps)`	Cache function identity
`useReducer`	`[s, dispatch] = useReducer(r, init)`	Complex state transitions
`useId`	`id = useId()`	Stable unique IDs for a11y
`useTransition`	`[pending, start] = useTransition()`	Mark updates non-urgent

Props vs State at a glance

Aspect	Props	State
Owner	Parent	The component itself
Mutable?	No (read-only)	Yes, via setter
Triggers re-render	When parent passes new values	When setter is called
Direction	Flows down	Local, raised via callbacks

Best Practices & Common Pitfalls

✓ **Do**

Keep components small and single-purpose. Derive values during render instead of storing redundant state. Co-locate state as low as possible. Name event handlers `handleX` and props `onX`.

✦ **Do**

Treat state as immutable, always producing new objects/arrays. Enable the ESLint React Hooks plugin and fix every exhaustive-deps warning.

⚠ **Avoid**

Storing derived data in state (e.g. a `fullName` computed from first/last) — it drifts out of sync. Compute it inline instead.

✕ **Avoid**

Calling hooks conditionally, mutating state/props directly, using array indexes as keys for dynamic lists, and putting `setState` in render (infinite loop). Don't use effects for logic that belongs in event handlers.

Show: the three most common beginner bugs

Interview Questions & Answers

What problem does the virtual DOM solve?

Direct DOM manipulation is slow and error-prone. React keeps a lightweight in-memory tree, computes the difference between the previous and next trees (reconciliation), and applies only the minimal set of real-DOM mutations. You write declarative code; React optimises the imperative updates.

What is the difference between props and state?

Props are read-only inputs passed from a parent and cannot be changed by the receiving component. State is private data owned and managed by the component itself, updated via its

setter, which schedules a re-render. Props flow down; state is local.

Why does React need keys in lists?

Keys give each element a stable identity so React can match items between renders and preserve their state/DOM when the list reorders or changes. Without stable keys (e.g. using array index), React may reuse the wrong DOM node, producing bugs like wrong input values. Keys should be stable, unique domain IDs.

Explain the useEffect dependency array.

It lists the reactive values the effect reads. React re-runs the effect whenever any dependency changes (by `Object.is` comparison). An empty array runs it once after mount; omitting the array runs it after every render. Dependencies must be complete — otherwise the effect captures stale values (stale closure bug).

Why must you not mutate state directly?

React detects changes by comparing references. Mutating an object/array keeps the same reference, so React (and memoised children) may skip re-rendering. Always create a new object/array (spread, map, filter) so the reference changes and the UI stays consistent.

What are the Rules of Hooks and why do they exist?

Call hooks only at the top level (not in loops/conditions/nested functions) and only from React functions. React associates each hook with its state by call order; conditional calls would misalign that order between renders and corrupt state.

useMemo vs useCallback — when do you use each?

`useMemo` caches the result of an expensive computation; `useCallback` caches a function's identity so it stays stable across renders (useful for props to memoised children or effect deps). `useCallback(fn, d)` equals `useMemo(() ⇒ fn, d)`. Use them to fix measured re-render problems, not pre-emptively.

What is "lifting state up"?

Moving shared state to the closest common ancestor of the components that need it, then passing the value down as props and raising changes via callback props. It keeps a single source of truth and enables sibling components to stay in sync.

Controlled vs uncontrolled components?

In a controlled component, form input value is driven by React state (`value` + `onChange`), making React the source of truth. An uncontrolled component lets the DOM hold the value, read on demand via a ref. Controlled gives instant validation/derived UI; uncontrolled is lighter for simple forms.

How do you prevent unnecessary re-renders?

Wrap pure components in `React.memo`, keep prop identities stable with `useCallback` / `useMemo`, use stable keys, move state down so changes affect smaller subtrees, and pass JSX via `children`. In 2026, the React Compiler can automate most of this. Always profile first.

What is reconciliation?

The algorithm React uses to diff the new element tree against the previous one. It assumes elements of different types produce different trees, and uses keys to match children of the same type. The result is a minimal list of DOM operations committed in a single batch.

Why can calling setState multiple times only update once, and how to fix it?

Within one render, state is a fixed snapshot, so `setCount(count + 1)` three times all read the same `count`. React also batches updates. Use the functional updater `setCount(c ⇒ c + 1)`, which receives the latest pending value, to apply sequential updates correctly.

Further Learning

- **Official docs** — react.dev, especially the "Learn React" and "Thinking in React" tutorials and the "Escape Hatches" chapter on effects and refs.
- **useReducer + Context** — scale state management before reaching for external libraries.
- **TanStack Query** — the standard for server-state caching and data fetching.
- **Next.js / Remix** — Server Components, streaming SSR, and full-stack React.
- **Testing** — React Testing Library + Vitest/Jest for user-centric component tests.
- **TypeScript with React** — typed props, hooks, and events for safer large codebases.
- **React Compiler** — adopt it to auto-optimise and simplify your components.